Slide 1 de 35

Aula 2 — Métodos

Dando comportamento às classes

Na aula passada, nossas classes tinham apenas **dados** (atributos). Hoje elas vão ganhar **ações** — os **métodos**. As entidades vão aprender a fazer coisas!

Aula 1 — O que tínhamos	Aula 2 — O que vamos ter
Classe com atributos	Classe com atributos E métodos
O cachorro TEM nome e raça	O cachorro TEM nome e raça E SABE latir e comer
O produto TEM preço e estoque	O produto TEM preço e estoque E SABE aplicar desconto
Dados estáticos	Dados + comportamento

Slide 2 de 35

Revisão Relâmpago

Testando a memória da aula anterior

Antes de avançar, vamos relembrar. Tente responder **sem olhar as anotações**:

1. O que é uma **classe**?
2. O que faz o **constructor**?
3. O que é o **this** dentro de uma classe?
4. Qual a diferença entre **classe** e **instância**?
5. O que acontece se eu adicionar um atributo novo a um objeto? A classe muda?

Desafio rápido (2 minutos): Crie uma classe `Filme` com os atributos `titulo`, `diretor`, `ano` e `nota` — de cabeça, sem consultar nada!

Slide 3 de 35

Revisão — Quiz

Conferindo a aula passada

Qual das alternativas descreve corretamente o que acontece ao executar `new Produto('Café', 10)`?

- Cria uma cópia da classe Produto
- ✓ Cria um novo objeto e executa o constructor com os argumentos 'Café' e 10 (Resposta Correta)
- Declara uma nova classe chamada Café
- Chama uma função chamada Produto

Explicação: Exato! O `new` cria uma nova instância (objeto) e roda automaticamente o `constructor`, passando os argumentos 'Café' e 10 como parâmetros.

Slide 4 de 35

O que São Métodos?

Funções que vivem dentro da classe

Métodos são funções definidas dentro de uma classe. Eles representam as **ações** que a entidade sabe executar. A grande diferença de uma função solta: o método tem acesso ao `this` — ele conhece seus próprios dados sem receber por parâmetro.

Quando `rex.latir()` é chamado, o `this` dentro do método é o `rex`. Quando `mel.latir()` é chamado, o `this` é a `mel`. Cada instância "sabe quem é" através do `this`.

Slide 5 de 35

O que Métodos Podem Fazer

Capacidades e características

- **Acessar atributos** da instância com `this.atributo`
- **Receber parâmetros** como qualquer função normal
- **Retornar valores** com `return` — ou não retornar nada
- **Modificar atributos** da instância (alterar o estado do objeto)
- **Chamar outros métodos** da mesma classe com `this.outroMetodo()`

Slide 6 de 35

Testando os Métodos do Cachorro

Vendo cada tipo de método em ação

```
const rex = new Cachorro("Rex", 20);

// Método que apenas executa
rex.latir(); // "Rex diz: Au au!"

// Método que retorna valor - posso usar em condições
console.log(rex.estaComFome()); // true (20 < 50)

if (rex.estaComFome()) {
  rex.comer(40); // modifica estado: energia 20 → 60
}

console.log(rex.estaComFome()); // false (60 < 50? Não!)

// Método que chama outros internamente
rex.rotinaDiaria();
// "Rex diz: Au au!"
// "Com fome? Não"
```

Slide 7 de 35

Verificação — Conceito de Métodos

Métodos vs funções soltas

Qual a principal vantagem de um método sobre uma função solta?

- O método é mais rápido
- ✓ O método tem acesso ao this — conhece seus próprios dados sem receber por parâmetro (Resposta Correta)
- O método pode receber parâmetros e a função não
- Não há diferença real — são a mesma coisa

Explicação: Exato! O método vive dentro da classe e acessa os dados da instância via `this`. Uma função solta precisa receber o objeto inteiro como parâmetro. Dados e comportamento ficam juntos no OO.

Slide 8 de 35

Exemplo Real — Conta Bancária

Métodos com validação e lógica de negócio

Vamos construir uma classe mais robusta. Uma `ContaBancaria` precisa de métodos que **validem** as operações antes de executar — não pode sacar mais do que tem, não pode depositar valor negativo.

Slide 9 de 35

Conta Bancária — Testando

Vendo os métodos em ação

```
const conta = new ContaBancaria("Ana", 1000);

conta.exibirExtrato();
// Titular: Ana | Saldo: R$1000.00

conta.depositar(500);
// Depósito de R$500.00. Saldo: R$1500.00

conta.sacar(200);
// Saque de R$200.00. Saldo: R$1300.00

conta.sacar(5000);
// Saldo insuficiente!

conta.depositar(-100);
// Erro: valor deve ser positivo.

conta.exibirExtrato();
// Titular: Ana | Saldo: R$1300.00
```

Slide 10 de 35

Métodos que Retornam vs Métodos que Executam

Uma distinção fundamental

Existe uma diferença importante entre métodos que **retornam um valor** e métodos que **apenas executam uma ação**. Misturar os dois é um erro comum de iniciantes.

Slide 11 de 35

Verificação — Return

Retorno de métodos

Se um método tem `console.log` mas NÃO tem `return`, o que ele retorna?

- O texto que foi impresso no console
- null
- ✓ undefined (Resposta Correta)
- false

Explicação: Correto! Funções e métodos sem `return` explícito retornam `undefined` implicitamente. O `console.log` apenas imprime na tela — não é um retorno.

Slide 12 de 35

Interação Entre Objetos — Composição

Uma classe usando instâncias de outra

O poder real do OO aparece quando objetos de classes diferentes **interagem entre si**. Uma classe pode armazenar e manipular instâncias de outra classe. Isso é chamado de **composição**.

O `CarrinhoDeCompras` não sabe criar produtos — ele só sabe **guardar e manipular**. Cada classe tem sua responsabilidade.

Slide 13 de 35

Carrinho de Compras — Implementação

Duas classes conversando

```
class Produto {
  constructor(nome = "Sem nome", preco = 0) {
    this.nome = nome;
```

```
    this.preco = preco;
  }
}

class CarrinhoDeCompras {
  constructor() {
    this.itens = [];
  }

  adicionarItem(produto, quantidade = 1) {
    this.itens.push({ produto, quantidade });
    console.log(`${produto.nome}
(x${quantidade}) adicionado ao carrinho.`);
  }

  removerItem(nomeProduto) {
    const tamanhoAntes = this.itens.length;
    this.itens = this.itens.filter(item
=> item.produto.nome !== nomeProduto);
    if (this.itens.length < tamanhoAntes) {
      console.log(`${nomeProduto} removido.`);
    } else {
      console.log(`${nomeProduto} não encontrado.`);
    }
  }

  calcularTotal() {
    return this.itens.reduce((total, item) => {
      return total + (item.produto.preco * item.quantidade);
    }, 0);
  }

  exibirResumo() {
    console.log("========= CARRINHO =========");
    if (this.itens.length === 0) {
      console.log("Carrinho vazio.");
      return;
    }
    this.itens.forEach(item => {
```

```
      const subtotal = item.produto.preco * item.quantidade;
      console.log(`${item.produto.nome}
x${item.quantidade} = R$${subtotal.toFixed(2)}`);
    });
    console.log(`============================`);
    console.log(`TOTAL: R$${this.calcularTotal().toFixed(2)}`);
  }
}
```

Slide 14 de 35

Carrinho de Compras — Testando

Vendo a composição em ação

```
const notebook = new Produto("Notebook", 3500);
const mouse = new Produto("Mouse", 89.90);
const teclado = new Produto("Teclado", 159.90);

const carrinho = new CarrinhoDeCompras();

carrinho.adicionarItem(notebook);
// "Notebook (x1) adicionado ao carrinho."

carrinho.adicionarItem(mouse, 2);
// "Mouse (x2) adicionado ao carrinho."

carrinho.adicionarItem(teclado);
// "Teclado (x1) adicionado ao carrinho."

carrinho.exibirResumo();
// ========= CARRINHO =========
// Notebook x1 = R$3500.00
// Mouse x2 = R$179.80
// Teclado x1 = R$159.90
// ============================
// TOTAL: R$3839.70

carrinho.removerItem("Mouse");
console.log(`Novo total:
R$${carrinho.calcularTotal().toFixed(2)}`);
// "Novo total: R$3659.90"
```

Slide 15 de 35

Método Chamando Método — this.outroMetodo()

Reutilização interna

Um método pode chamar outro da mesma classe com `this.nomeDoMetodo()`.
Isso promove reutilização — se a lógica muda, só altera em um lugar.

Slide 16 de 35

Transferência — Testando

Dois objetos da mesma classe interagindo

```
const contaAna = new ContaBancaria("Ana", 1000);
const contaBia = new ContaBancaria("Bia", 500);

contaAna.transferir(300, contaBia);
// "R$300: Ana → Bia"

console.log(contaAna.saldo); // 700
console.log(contaBia.saldo); // 800

contaAna.transferir(5000, contaBia);
// "Transferência falhou."

console.log(contaAna.saldo); // 700 — inalterado
```

O `transferir` não reimplementa a lógica de saque e depósito — ele **reutiliza** os métodos que já existem. Se a regra de saque mudar no futuro, basta alterar em um lugar.

Slide 17 de 35

Verificação — Interação

Objetos interagindo

No método `transferir(valor, contaDestino)`, o que representa `contaDestino`?

Uma string com o nome do titular

✓ Uma instância de ContaBancaria — um objeto completo com seus métodos (Resposta Correta)

O saldo da conta destino

A classe ContaBancaria em si

Explicação: Exato! `contaDestino` é uma instância de `ContaBancaria` — um objeto completo com todos os atributos e métodos. Por isso podemos chamar `contaDestino.depositar(valor)` dentro do método.

Slide 18 de 35

Vocês Já Usam Métodos Todos os Dias!

O que vocês já conhecem sem saber

Se vocês já programam em JavaScript, **já usam métodos o tempo todo** sem perceber. Tudo que chamam com `.algumaCoisa()` é um método de alguma classe ou objeto:

Código que vocês já usam	O que realmente é
`"hello".toUpperCase()`	Método da classe String
`[1,2,3].push(4)`	Método da classe Array
`[1,2,3].filter(n => n > 1)`	Método da classe Array
`console.log("oi")`	Método do objeto console
`document.getElementById("x")`	Método do objeto document
`Math.random()`	Método do objeto Math
`JSON.stringify(obj)`	Método do objeto JSON
`new Date().getFullYear()`	Método da classe Date

Slide 19 de 35

Estudo de Caso — Formas Geométricas com Métodos

Retomando as classes da Aula 1

Vamos retomar as formas geométricas da aula passada e adicionar **comportamento**. Agora elas vão saber calcular área, perímetro e exibir informações.

Slide 20 de 35

Círculo com Métodos

Implementação completa

```
const c1 = new Circulo(5);
c1.exibirInfo();
// Círculo (raio=5):
//   Área: 78.54
//   Perímetro: 31.42

// Posso usar o retorno em cálculos
const areaTotal = c1.calcularArea()
+ new Circulo(3).calcularArea();
console.log(areaTotal.toFixed(2)); // 106.81
```

Slide 21 de 35

Retângulo e Triângulo

Expandindo o exemplo

```javascript
class Retangulo {
  constructor(base = 1, altura = 1) {
    this.base = base;
    this.altura = altura;
  }

  calcularArea() {
    return this.base * this.altura;
  }

  calcularPerimetro() {
    return 2 * (this.base + this.altura);
  }

  ehQuadrado() {
    return this.base === this.altura;
  }

  exibirInfo() {
    const tipo = this.ehQuadrado() ? "Quadrado" : "Retângulo";
    console.log(`${tipo} (${this.base}x${this.altura}):`);
    console.log(`  Área: ${this.calcularArea()}`);
    console.log(`  Perímetro: ${this.calcularPerimetro()}`);
  }
}
```

```
class Triangulo {
  constructor(ladoA = 1, ladoB = 1,
ladoC = 1, base = 1, altura = 1) {
    this.ladoA = ladoA;
    this.ladoB = ladoB;
    this.ladoC = ladoC;
    this.base = base;
    this.altura = altura;
  }

  calcularArea() {
    return (this.base * this.altura) / 2;
  }

  calcularPerimetro() {
    return this.ladoA + this.ladoB + this.ladoC;
  }

  ehEquilatero() {
    return this.ladoA === this.ladoB
&& this.ladoB === this.ladoC;
  }

  exibirInfo() {
    const tipo = this.ehEquilatero()
? "Equilátero" : "Triângulo";
    console.log(`${tipo} (lados: ${this.ladoA},
${this.ladoB}, ${this.ladoC}):`);
    console.log(`  Área: ${this.calcularArea()}`);
    console.log(`  Perímetro: ${this.calcularPerimetro()}`);
  }
}
```

Slide 22 de 35

Comparando Objetos — Método que Recebe Outra Instância

Um objeto pode receber outro como parâmetro

Um conceito poderoso: um método pode receber **outra instância** como parâmetro e interagir com ela. Isso funciona inclusive entre classes diferentes.

```
class Circulo {
  constructor(raio = 1) {
    this.raio = raio;
  }

  calcularArea() {
    return Math.PI * this.raio ** 2;
  }

  // Recebe QUALQUER objeto que tenha calcularArea()
  compararCom(outraForma) {
    const minhaArea = this.calcularArea();
    const outraArea = outraForma.calcularArea();

    if (minhaArea > outraArea) {
      return "Eu sou maior.";
    } else if (minhaArea < outraArea) {
      return "O outro é maior.";
    }
    return "Áreas iguais.";
  }
}

const circulo = new Circulo(5);
const retangulo = new Retangulo(10, 8);

console.log(circulo.compararCom(retangulo));
// "O outro é maior." (78.54 vs 80)
```

Slide 23 de 35

Verificação — Instâncias como Parâmetro

Objetos como parâmetros de métodos

No método `compararCom(outraForma)`, por que funciona com Circulo, Retangulo e Triangulo?

- Porque são todas a mesma classe
- ✓ Porque todas são instâncias que possuem o método calcularArea() (Resposta Correta)
- Porque o JavaScript converte automaticamente
- Porque herdam de uma classe base Forma

Explicação: Correto! O que importa não é o TIPO da classe, mas se o objeto tem o método `calcularArea()`. Como todas as formas implementam esse método, todas funcionam. Isso é a base do polimorfismo — Aula 4!

Slide 24 de 35

Exemplo Real — Sistema de Avaliações

Caso de uso com métodos diversos

Vamos ver um exemplo mais rico: um sistema de avaliação de restaurante que combina vários array methods dentro de uma classe.

```
class Restaurante {
  constructor(nome, tipo) {
    this.nome = nome;
    this.tipo = tipo;
    this.avaliacoes = [];
  }

  adicionarAvaliacao(nota, comentario = "") {
    if (nota < 1 || nota > 5) {
```

```
      console.log("Nota deve ser entre 1 e 5.");
      return;
    }
    this.avaliacoes.push({ nota,
comentario, data: new Date() });
  }

  calcularMedia() {
    if (this.avaliacoes.length === 0) return 0;
    const soma = this.avaliacoes.reduce((acc,
av) => acc + av.nota, 0);
    return soma / this.avaliacoes.length;
  }

  obterEstrelas() {
    return "⭐".repeat(Math.round(this.calcularMedia()));
  }

  obterMelhoresComentarios() {
    return this.avaliacoes
      .filter(av => av.nota >= 4 && av.comentario)
      .map(av => av.comentario);
  }

  exibirResumo() {
    console.log(`${this.nome} (${this.tipo})`);
    console.log(`Nota: ${this.calcularMedia().toFixed(1)}
${this.obterEstrelas()}`);
    console.log(`Total de avaliações:
${this.avaliacoes.length}`);
  }
}
```

Slide 25 de 35

Sistema de Avaliações — Testando

Classe Restaurante em ação

```
const sushi = new Restaurante("Tokyo Sushi", "Japonesa");

sushi.adicionarAvaliacao(5, "Melhor sushi da cidade!");
sushi.adicionarAvaliacao(4, "Muito bom, mas demorou.");
sushi.adicionarAvaliacao(5, "Perfeito!");
sushi.adicionarAvaliacao(3);
sushi.adicionarAvaliacao(4, "Ótimo custo-benefício.");
sushi.adicionarAvaliacao(8); // "Nota
deve ser entre 1 e 5." - rejeitado!

sushi.exibirResumo();
// Tokyo Sushi (Japonesa)
// Nota: 4.2 ⭐⭐⭐⭐
// Total de avaliações: 5

console.log(sushi.obterMelhoresComentarios());
// ["Melhor sushi da cidade!", "Muito bom, mas demorou.",
//  "Perfeito!", "Ótimo custo-benefício."]
```

Esse exemplo usa `reduce` para somar notas, `filter` + `map` para obter comentários, `repeat` para estrelas, `toFixed` para formatar. Os array methods que vocês já conhecem agora vivem **organizados dentro de uma classe**.

Slide 26 de 35

Boas Práticas com Métodos

Escrevendo métodos profissionais

Slide 27 de 35

Boas Práticas — Código Comparado

Antes e depois aplicando as regras

Slide 28 de 35

Verificação — Boas Práticas

Identificando código limpo

Por que é melhor um método `calcularTotal()` que RETORNA o valor em vez de apenas imprimi-lo?

- Porque console.log é lento
- ✓ Porque retornando, o resultado pode ser armazenado, usado em cálculos, em condições ou passado para outro método (Resposta Correta)
- Porque métodos não podem usar console.log
- Não é melhor — ambos são equivalentes

Explicação: Correto! Um método que retorna é reutilizável: `const total = calcularTotal(); if (total > 100) aplicarDesconto();`. Um que só imprime fica limitado à exibição.

Slide 29 de 35

Exemplo Completo — Gerenciador de Contatos

CRUD em miniatura com métodos

Para consolidar, vamos ver um exemplo com **CRUD** (Create, Read, Update, Delete). Esse padrão será a base do To-Do List nas aulas 5-6.

Slide 30 de 35

Gerenciador de Contatos — Implementação

Código das duas classes

```
class Contato {
  constructor(nome, telefone, email = "") {
    this.nome = nome;
    this.telefone = telefone;
    this.email = email;
  }

  exibir() {
    let info = `${this.nome} – ${this.telefone}`;
    if (this.email) info += ` – ${this.email}`;
    return info;
  }
}

class Agenda {
  constructor() {
    this.contatos = [];
  }

  // CREATE
  adicionar(nome, telefone, email) {
    const contato = new Contato(nome, telefone, email);
    this.contatos.push(contato);
    return contato;
  }

  // READ
  buscarPorNome(termo) {
    return this.contatos.filter(c =>
      c.nome.toLowerCase().includes(termo.toLowerCase())
    );
  }

  listarTodos() {
    if (this.contatos.length === 0) {
      console.log("Agenda vazia.");
      return;
    }
```

```
    this.contatos.forEach((c, i) => {
      console.log(`${i + 1}. ${c.exibir()}`);
    });
  }

  // UPDATE
  atualizarTelefone(nome, novoTelefone) {
    const contato = this.contatos.find(c => c.nome === nome);
    if (!contato) {
      console.log(`${nome} não encontrado.`);
      return false;
    }
    contato.telefone = novoTelefone;
    return true;
  }

  // DELETE
  remover(nome) {
    const tamanho = this.contatos.length;
    this.contatos = this.contatos.filter(c => c.nome !== nome);
    return this.contatos.length < tamanho;
  }

  totalContatos() {
    return this.contatos.length;
  }
}
```

Slide 31 de 35

Gerenciador de Contatos — Testando o CRUD

Create, Read, Update, Delete em ação

```
const agenda = new Agenda();

// CREATE
agenda.adicionar("Ana Silva", "
(45) 99999-1111", "ana@email.com");
agenda.adicionar("Carlos Souza", "(45) 99999-2222");
agenda.adicionar("Ana Costa", "(45)
99999-3333", "anacosta@email.com");

// READ – listar
agenda.listarTodos();
// 1. Ana Silva – (45) 99999-1111 – ana@email.com
// 2. Carlos Souza – (45) 99999-2222
// 3. Ana Costa – (45) 99999-3333 – anacosta@email.com

// READ – buscar
const anas = agenda.buscarPorNome("ana");
console.log(`Encontrados: ${anas.length}`); // 2

// UPDATE
agenda.atualizarTelefone("Carlos Souza", "(45) 88888-0000");

// DELETE
agenda.remover("Ana Costa");
console.log(`Total: ${agenda.totalContatos()}`); // 2
```

Slide 32 de 35

Atividade de Fixação 2

Adicionando métodos às formas geométricas

Atividade prática (escopo fechado): Retome as classes de formas geométricas da Aula 1 e adicione métodos.

1. Adicione pelo menos **2 métodos de cálculo** para cada forma: `calcularArea()` e `calcularPerimetro()`
2. Adicione um método `exibirInfo()` que mostra todos os dados formatados (use template literals e `toFixed(2)`)
3. Os métodos de cálculo devem **RETORNAR** o valor (não apenas imprimir)
4. O `exibirInfo()` deve **CHAMAR** os métodos de cálculo internamente (`this.calcularArea()`)
5. **Bônus:** Crie um método `compararArea(outraForma)` que recebe outra instância e diz qual tem maior área

```
// Fórmulas para referência:

// Círculo:
//   Área = π × raio²        → Math.PI * this.raio ** 2
//   Perímetro = 2 × π × raio → 2 * Math.PI * this.raio

// Retângulo:
//   Área = base × altura             → this.base * this.altura
//   Perímetro = 2 × (base + altura)
→ 2 * (this.base + this.altura)

// Triângulo:
//   Área = (base × altura) / 2
→ (this.base * this.altura) / 2
//   Perímetro = ladoA + ladoB + ladoC
→ this.ladoA + this.ladoB + this.ladoC
```

Slide 33 de 35

Lição de Casa — Aula 2

Playlist com métodos completos

Para entregar: Crie duas classes que interagem entre si: `Musica` e `Playlist`.

- `Musica` — atributos: titulo, artista, duracaoSegundos.
  Método `duracaoFormatada()` retorna em `mm:ss`.
  Método `exibir()` retorna string formatada
- `Playlist` — atributo `musicas` começa como array vazio
- `adicionarMusica(musica)` — recebe instância de Musica e adiciona ao array
- `removerMusica(titulo)` — remove pelo título
- `exibirPlaylist()` — mostra todas as músicas numeradas com duração
- `calcularDuracaoTotal()` — retorna a soma
  de todas as durações em formato `mm:ss`
- `buscarMusica(titulo)` — retorna a música encontrada ou mensagem
- Testar com **pelo menos 5 músicas**

```
// Dica para formatar mm:ss:
function formatarTempo(totalSegundos) {
  const min = Math.floor(totalSegundos / 60);
  const seg = totalSegundos % 60;
  return `${min}:${String(seg).padStart(2, "0")}`;
}
// formatarTempo(185) → "3:05"
// formatarTempo(60)  → "1:00"
```

Slide 34 de 35

Resumo da Aula 2

Revisão ativa com flashcards

Teste sua memória — tente responder antes de virar cada cartão:

Slide 35 de 35

O que Vem na Aula 3?

Prévia do Encapsulamento

Hoje nossas classes ganharam **comportamento** com métodos. Mas um problema ficou pendente:

Na **Aula 3 — Encapsulamento**, vamos resolver isso com **campos privados** (`#atributo`), **getters** e **setters**. Os dados serão realmente protegidos!

Total de 35 slides